UNIVERSITY OF CAMBRIDGE

MPhil in Data Intensive Science

Michaelmas Term 2025

C1 Research Computing Coursework

B. Bolliet

*Attempt **all** parts of the coursework. Code is submitted to your individual GitLab repository. The repository will be assessed after 17 December 2025.*

Include:

- *Link to your GitLab project repository.*

*There are **135 marks** in total.*

© 2025 University of Cambridge

DRAFT 2

Preamble

Your goal is to build a full-stack, research-grade system capable of learning and serving neural-network models that can interpolate any 5-dimensional numerical dataset. A test dataset is provided.
The system must have 6 components:

1. A **Python package** implementing data loading, model training, and inference.
2. A **ReadTheDocs** html build (local).
3. A **FastAPI backend** exposing REST endpoints for training and prediction.
4. A **Next.js frontend** providing an interactive UI to train and query models.
5. A **GitLab** repository containing all of the above. The commit history should reflect your answers to each of the questions (use a different tag for each question).
6. A **Docker-compose** file to serve the app in a container.

Tasks

1 Create repository structure [5]

Create a Git repository named `interpolator` with this overall structure (your repository will eventually contain more files):

```
.
|-- backend/
|   |-- pyproject.toml
|   |-- fivedreg
|       |-- main.py
|-- frontend/
    |-- package.json
    |-- src/app/
```

2 Package configuration [5]

Configure the backend with a valid `pyproject.toml` (PEP 621). Declare dependencies (e.g., `numpy`, `torch`, `fastapi`, `uvicorn`, etc.).

3 Data handling [10]

Implement a `fivedreg.data` module to handle 5D datasets from `.pkl` format. The module should include a `load_dataset(filepath)` function that reads X (5 features) and y (target) arrays, validates input dimensions and handles missing values appropriately. Implement functionality to split data into train/validation/test sets and standardizes features. Your implementation should handle the test dataset provided.

© 2025 University of Cambridge

4 **Implement model** [15]

Implement a lightweight neural network torch, scikit-learn or tensorflow. The model should use small hidden layers (e.g., [64, 32, 16]) and be fully configurable by number of layers, neurons per layer, learning rate, and max iterations. Performance is critical: the model must train on CPU within less than a minute for datasets up to 10,000 samples. Ensure the implementation provides a consistent `fit(X, y)` and `predict(X)` interface compatible with torch/tensorflow conventions. Your implementation should work with the dataset provided.

5 **FastAPI backend** [20]

Create a comprehensive FastAPI backend that handles dataset upload and model operations. The API should provide a `GET /health` endpoint for service health checks, a `POST /upload` endpoint for uploading 5D datasets in `.npz` format, and a `POST /train` endpoint that trains models on uploaded datasets. Additionally, implement a `POST /predict` for making predictions on new 5D input vectors.

6 **Next.js frontend** [20]

Develop a modern, responsive Next.js frontend that provides an intuitive interface for the machine learning system. The application should feature four main pages: an upload page (`/upload`) with a file upload interface for `.npz` datasets including validation and preview functionality, a training button (`/train`) where users can configure hyperparameters and start training, a prediction button (`/predict`) with an interactive form containing 5 input fields for making predictions. The result of the prediction should display.

7 **Testing and reproducibility** [20]

Create a comprehensive testing suite under `backend/tests/` that validates all system components and ensures reliability. Additionally, implement containerization using `Dockerfile` configurations and a `docker-compose.yml` file that can launch both backend and frontend services seamlessly. The implementation should include thorough documentation of all environment variables, usage instructions, and setup procedures in a detailed `README.md` file that enables easy deployment and development workflow.

8 **Performance and profiling** [20]

Conduct comprehensive performance analysis and benchmarking of the model to understand its computational characteristics and scalability. This analysis should measure training time as a function of dataset size using datasets of 1K, 5K, and 10K samples to establish scaling behavior. Additionally, profile memory usage during both training and prediction phases to identify potential bottlenecks, and compare accuracy metrics such as MSE and R^2 scores across different dataset sizes. You should report on your findings in the documentation page (next question), in a section called "Performances and profiling".

© 2025 University of Cambridge

9 **Documentation and deployment** [20]

Create comprehensive documentation using Sphinx and establish a complete local deployment strategy. The documentation system should use Sphinx to generate HTML documentation that includes detailed API reference, user guides, installation instructions, and practical usage examples, as well as a description of the test suite. All documentation must build locally and be accessible through standard web browsers via file:// URLs. You should provide a shell script to build the documentation. Provide a separate shell script that successfully launches the entire technology stack locally. User should be able to upload the dataset provided.

END OF PAPER

© 2025 University of Cambridge